

SAPUI5

Adrian Streitz, Johannes Rank, Borys Levkovskyi



Prerequisites

- Fundamental programming skills
- Successful completion of the chapter *Introduction to SAP Application Development*
- Recommended: Completion of the chapter *HTML5, CSS & JavaScript*

Agenda

- I. Fundamentals**
- II. Creating and implementing SAPUI5 applications**
- III. Model-View-Controller**
- IV. Modularization & Navigation**
- V. Debugging SAPUI5 applications**

SAPUI5

- Toolkit for developing client applications for mobile and desktop platforms
- Based on HTML5, JavaScript, jQuery and the Model-View-Controller (MVC) concept
- Contains a large library of simple and complex UI controls
- Allows to create flexible and portable applications
- SAPUI5 applications can be run on several platforms (SAP NetWeaver AS ABAP, SAP HANA Cloud Platform, SAP Mobile Platform)

Anatomy of an SAPUI5 application

- SAPUI5 applications consist of view/controller pairs which define layout and logic of the application
- Every application has an initial view/controller pair
- Central index.html file consisting of the following sections:
 - Theme definition and including of the SAPUI5 libraries
 - Component Container instantiation
 - Id for the <body>, into which the application will be embedded

```
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta http-equiv='Content-Type' content='text/html; charset=UTF-8' />
    <title>SAPUI5 Navigation</title>
    <script
      src="resources/sap-ui-core.js"
      id="sap-ui-bootstrap"
      data-sap-ui-theme="sap_bluecrystal"
      data-sap-ui-libs="sap.m"
      data-sap-ui-bindingSyntax="complex"
      data-sap-ui-compatVersion="edge"
      data-sap-ui-preload="async"
      data-sap-ui-resourceroots='{ "mvc_app_000": "/" }'
    >
    </script>
    <script>
      sap.ui.getCore().attachInit(function () {
        sap.ui.require([
          "sap/m/Shell",
          "sap/ui/core/ComponentContainer"
        ], function (Shell, ComponentContainer) {
          new Shell({
            app : new ComponentContainer({
              name : "mvc_app_000",
              settings : {
                id : "webapp"
              }
            })
          }).placeAt("content");
        });
      });
    </script>
  </head>
  <body class="sapUiBody" id="content">
  </body>
</html>
```

Namespace
Definition

Including themes
and libraries

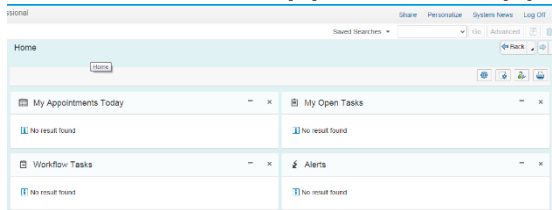
Instantiating a
Component
Container

Default content
section

II. Creating and implementing SAPUI5 applications

Index.html

- Main entry point for every SAPUI5 Application
- **Meta information:** document mode, charset
- **Bootstrap:** application runtime configuration
 - **src:** path to UI5 source files (local or CDN)
 - **theme:** defines the application appearance (bluecrystal, gold reflection, etc)
 - **libs:** `sap.m` or `sap.ui.commons`
 - **bindingSyntax:** determines the syntax for the data binding in the XML views - *default* or *complex*; complex should be used since the 1.28 version
 - **compatVersion:** set to *edge* to assure the most recent functionality of UI5
 - **preload:** *async* - UI5 resources loaded simultaneously in the background



First Name	Last Name	Sicil No
Eratper	Yilmaz	1
Anakin	Skywalker	2
Kit	Walker	3
Kodraz	Kodraz	4

```
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta http-equiv='Content-Type' content='text/html; charset=UTF-8' />
    <title>SAPUI5 Navigation</title>
    <script
      src="resources/sap-ui-core.js"
      id="sap-ui-bootstrap"
      data-sap-ui-theme="sap_bluecrystal"
      data-sap-ui-libs="sap.m"
      data-sap-ui-bindingSyntax="complex"
      data-sap-ui-compatVersion="edge"
      data-sap-ui-preload="async"
      data-sap-ui-resourceroots='{ "mvc_app_000": "/" }'
    >
    </script>
    <script>
      sap.ui.getCore().attachInit(function () {
        sap.ui.require([
          "sap/m/Shell",
          "sap/ui/core/ComponentContainer"
        ], function (Shell, ComponentContainer) {
          new Shell({
            app : new ComponentContainer({
              name : "mvc_app_000",
              settings : {
                id : "webapp"
              }
            })
          }).placeAt("content");
        });
      });
    </script>
  </head>
  <body class="sapUiBody" id="content">
  </body>
</html>
```

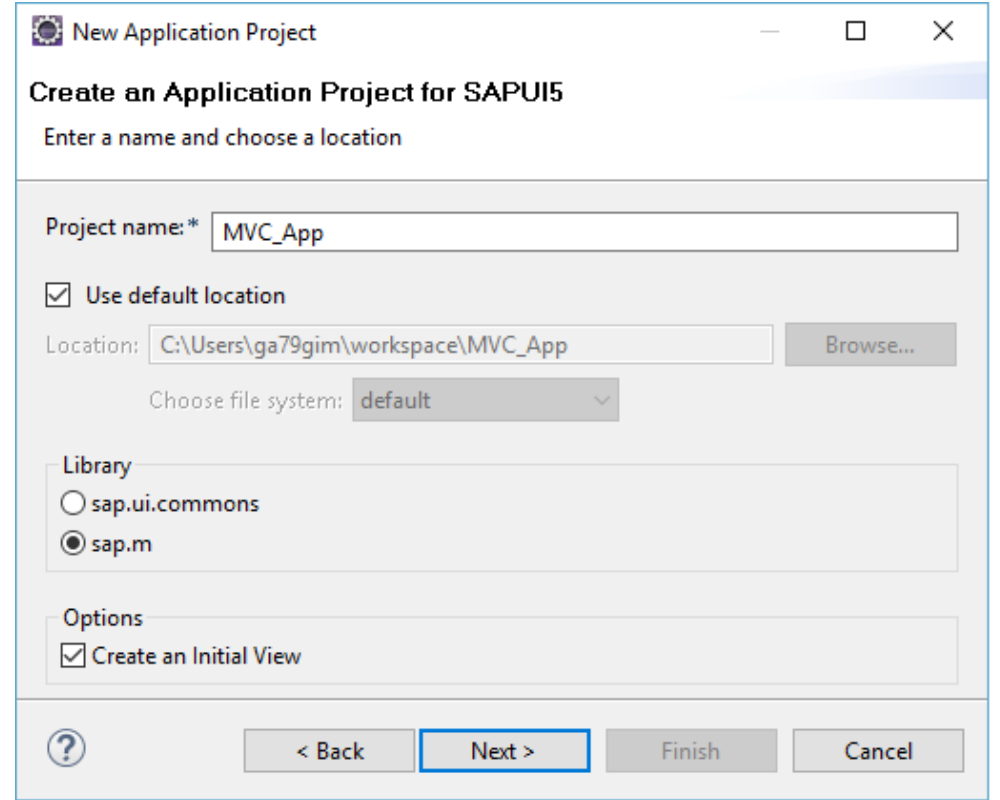
Meta information

Bootstrap

II. Creating and implementing SAPUI5 applications

Step 1: Create a new SAPUI5 application project

- In the Project Explorer, choose *New -> Other -> SAPUI5 Application Development -> Application Project*
- It is recommended to create an initial view
- You can choose between different formats for your initial view
- Creating an initial view automatically creates a controller for this view



The screenshot shows the 'New Application Project' dialog box. The title bar says 'New Application Project'. The main heading is 'Create an Application Project for SAPUI5'. Below it, it says 'Enter a name and choose a location'. The 'Project name:' field contains 'MVC_App'. The 'Use default location' checkbox is checked. The 'Location:' field shows 'C:\Users\ga79gim\workspace\MVC_App' with a 'Browse...' button. The 'Choose file system:' dropdown is set to 'default'. Under the 'Library' section, 'sap.m' is selected with a radio button, while 'sap.ui.commons' is unselected. Under the 'Options' section, the 'Create an Initial View' checkbox is checked. At the bottom, there are buttons for '?', '< Back', 'Next >', 'Finish', and 'Cancel'. The 'Next >' button is highlighted with a blue border.

II. Creating and implementing SAPUI5 applications

Step 2: Implement view-controller pairs

- View and controller are always implemented together
- The view defines the layout elements, while the controller defines the logic and event handling for the view
- SAPUI5 has a huge library of predefined view elements which can be used for the implementation
- A huge collection of code examples for the different view elements can be accessed via <https://sapui5.hana.ondemand.com/>

The screenshot shows the SAPUI5 IDE interface. On the left, a sidebar lists various UI elements under the 'Entities (209)' tab. The 'Icon Tab Bar' element is selected, showing a count of 7. The main editor area displays the code for the 'Icon Tab Bar - Filter' example. The code is split into two tabs: 'IconTabBar.view.xml' and 'IconTabBar.controller.js'. The XML code defines the view structure, including the 'IconTabBar' control and its 'items'. The JavaScript code implements the controller logic, including the 'handleIconTabBarSelect' method and the 'onInit' method. The code is as follows:

```
<code><?xml version='1.0' encoding='utf-8'?>
<mvc:View
  controllerName="sap.m.sample.IconTabBar.IconTabBar"
  xmlns:mvc="sap.ui.core.mvc"
  xmlns="sap.m">
  <IconTabBar
    id="idIconTabBar"
    select="handleIconTabBarSelect"
    class="sapUiResponsiveContentPadding">
    <items>
      <IconTabFilter
        showAll="true"
        count="{/ProductCollectionStats/Counts/Total}"
        text="Products"
        keys="All" />
      <IconTabSeparator />
      <IconTabFilter
        icon="sap-icon://begin"
        iconColor="Positive"
        count="{/ProductCollectionStats/Counts/Weight/Ok}"
        text="Ok"
        key="Ok" />
      <IconTabFilter
        icon="sap-icon://compare"
        iconColor="Critical"
        count="{/ProductCollectionStats/Counts/Weight/Heavy}"
        text="Heavy"
        key="Heavy" />
      <IconTabFilter
        icon="sap-icon://inventory"
        iconColor="Negative"
        count="{/ProductCollectionStats/Counts/Weight/Overweight}"
        text="Overweight"
        key="Overweight" />
    </items>
  </IconTabBar>
</mvc:View>

// Controller
define([
  "sap.m.sample.IconTabBar",
  "sap.m.sample.IconTabBar.view"
], function (IconTabBar, view) {
  "use strict";

  return {
    onInit: function () {
      this.getView().setModel(this.getModel());
    },
    handleIconTabBarSelect: function (oEvent) {
      // ...
    }
  };
});</code>
```

II. Creating and implementing SAPUI5 applications

Step 3: Deploy the application in the SAP backend

- In the Project Explorer, choose with a right click *Team -> Share Project*
- Choose the *SAPUI5 ABAP Repository* as repository object
- Connect to the SAP system to which you want to deploy the application
- Create a new BSP application as container for your SAPUI5 implementation
- You can repeatedly submit implementation changes to the SAP backend via *Team -> Submit*

Share Project

SAPUI5 Repository

Specify a BSP application

☐ Select a BSP Application:

Name	Description
/UI2/LAUNCHPAGE	Launchpage for Tile CHIPS
/UI2/TILECHIPS	CHIPS that can be used in Launchpage
/UI2/USHELL	Unified Shell
/UI2/USHELL_GWA	Generic Wrapper Application
/UI2/USHELL_ME	UShell Me Profile
/UI5/SIMPLETEST	Hello World
AOC	ABAP Online Community - SAPUI5 Application
ARSRVC_LPD_C	Actions Reusable CHIP
ARSRVC_SM	Social Media Productivity Services
ARSRVC_SPB_ADMIN	Suite Page Builder - Administration
ARSRVC_SUITE_PB	Suite Page Builder
ARSRVC_UPB_ADMIN	Unified Shell Page Builder Administration Page
AR_SRVC_NEWS	News Tile
BPCWEBCLIENT	BPC universal web client

BSP Application:

☒ Create a new BSP Application

Name *

Description *

Package *

Note:
Consider security aspects when using services via the Internet/Intranet, especially for administrative and development tasks.
For more information about secure handling of administrative tasks, see SAP Notes 1778777 and 1748112.

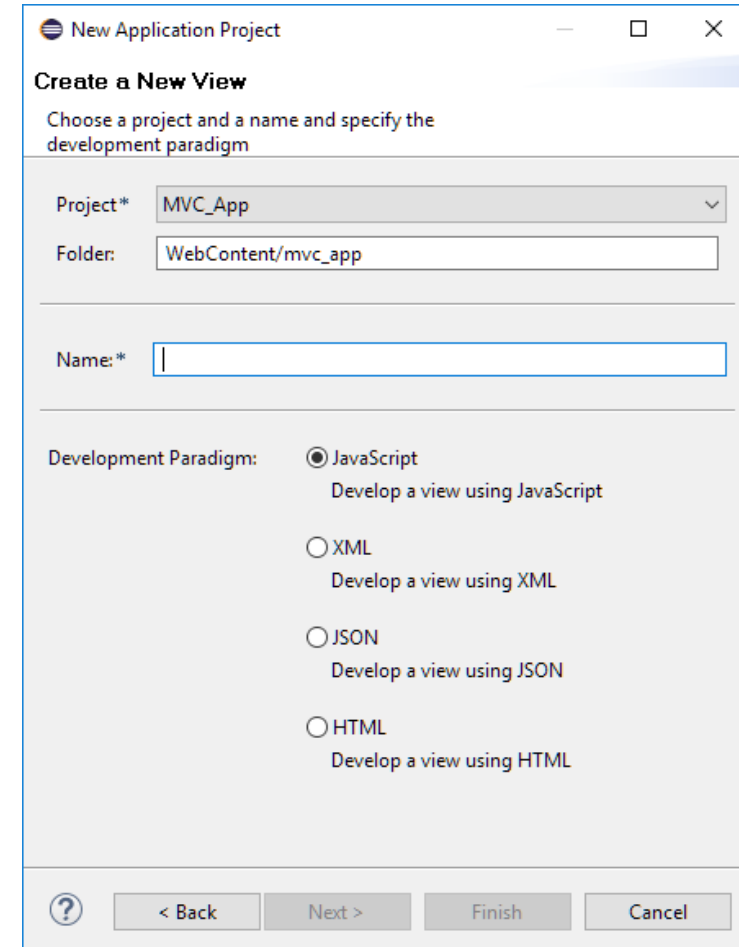
Architectural pattern

- Model:
 - Manages data and connections to data sources
 - Holds application data
- View:
 - Creates user interface
 - Passes user actions to a separate controller
- Controller:
 - Contains control logic
 - Manages interaction between view and model
 - Communicates with the model if a user action evokes data operations

View types

Views

- Views can be implemented in different forms:
 - **JavaScript:** sap.ui.core.mvc.JSView
 - **XML:** sap.ui.core.mvc.XMLView
 - **JSON:** sap.ui.core.mvc.JSONView
 - **HTML:** sap.ui.core.mvc.HTMLView
- Creating views using XML has several advantages:
 - Clearer implementation
 - XML can be checked and validated easier
 - SAP Fiori is completely based on XML views



The screenshot shows a 'New Application Project' dialog box with the 'Create a New View' tab selected. The dialog prompts the user to 'Choose a project and a name and specify the development paradigm'. The 'Project*' field is set to 'MVC_App' and the 'Folder' is 'WebContent/mvc_app'. The 'Name:*' field is empty. Under 'Development Paradigm', four options are listed: JavaScript (selected), XML, JSON, and HTML. Each option has a sub-label: 'Develop a view using JavaScript', 'Develop a view using XML', 'Develop a view using JSON', and 'Develop a view using HTML'. At the bottom, there are buttons for '?', '< Back', 'Next >', 'Finish', and 'Cancel'.

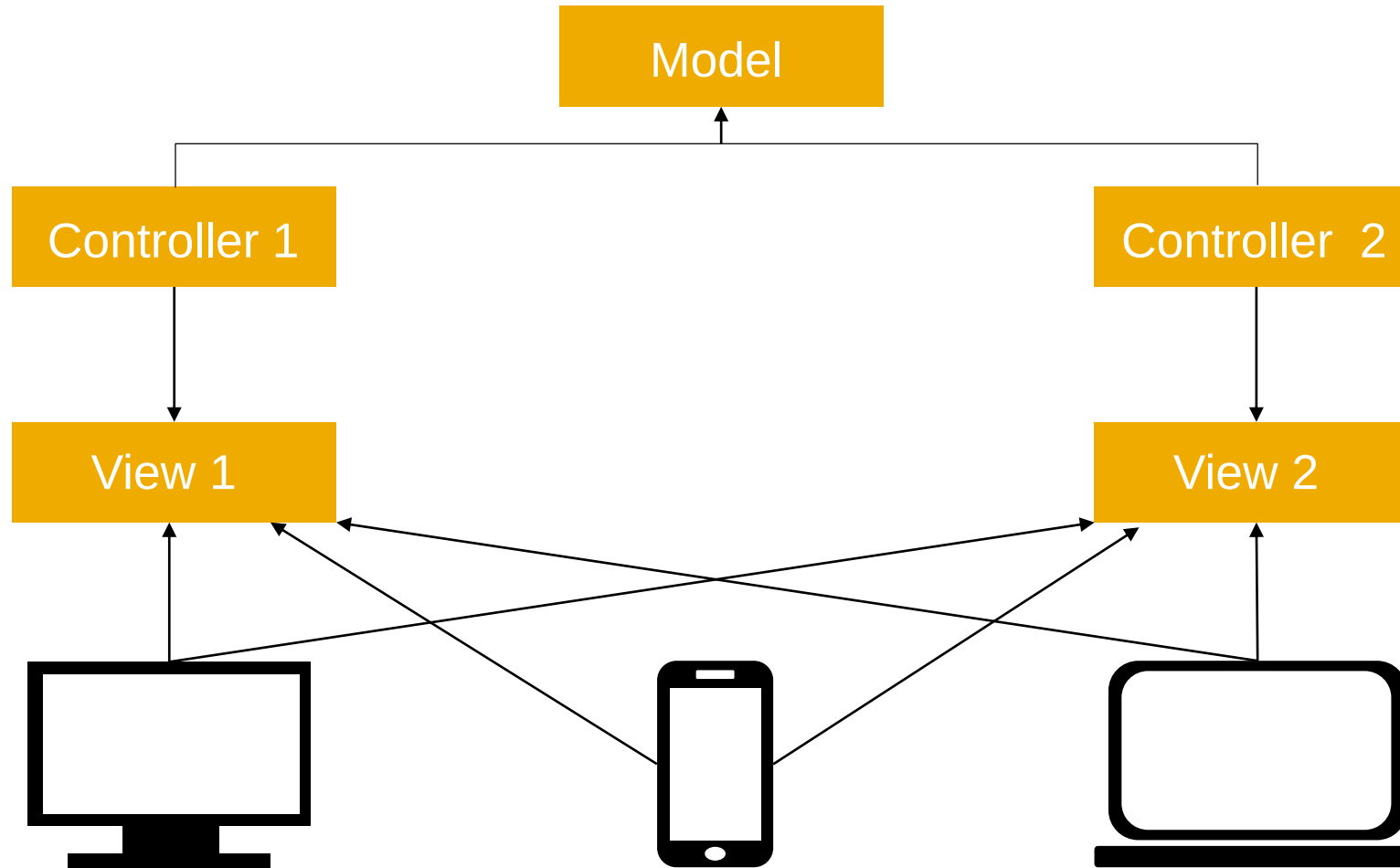
Now you know the fundamentals of SAPUI5 applications.

To consolidate your knowledge, you can do task 1, 2 and 3 of the SAPUI5 exercise.



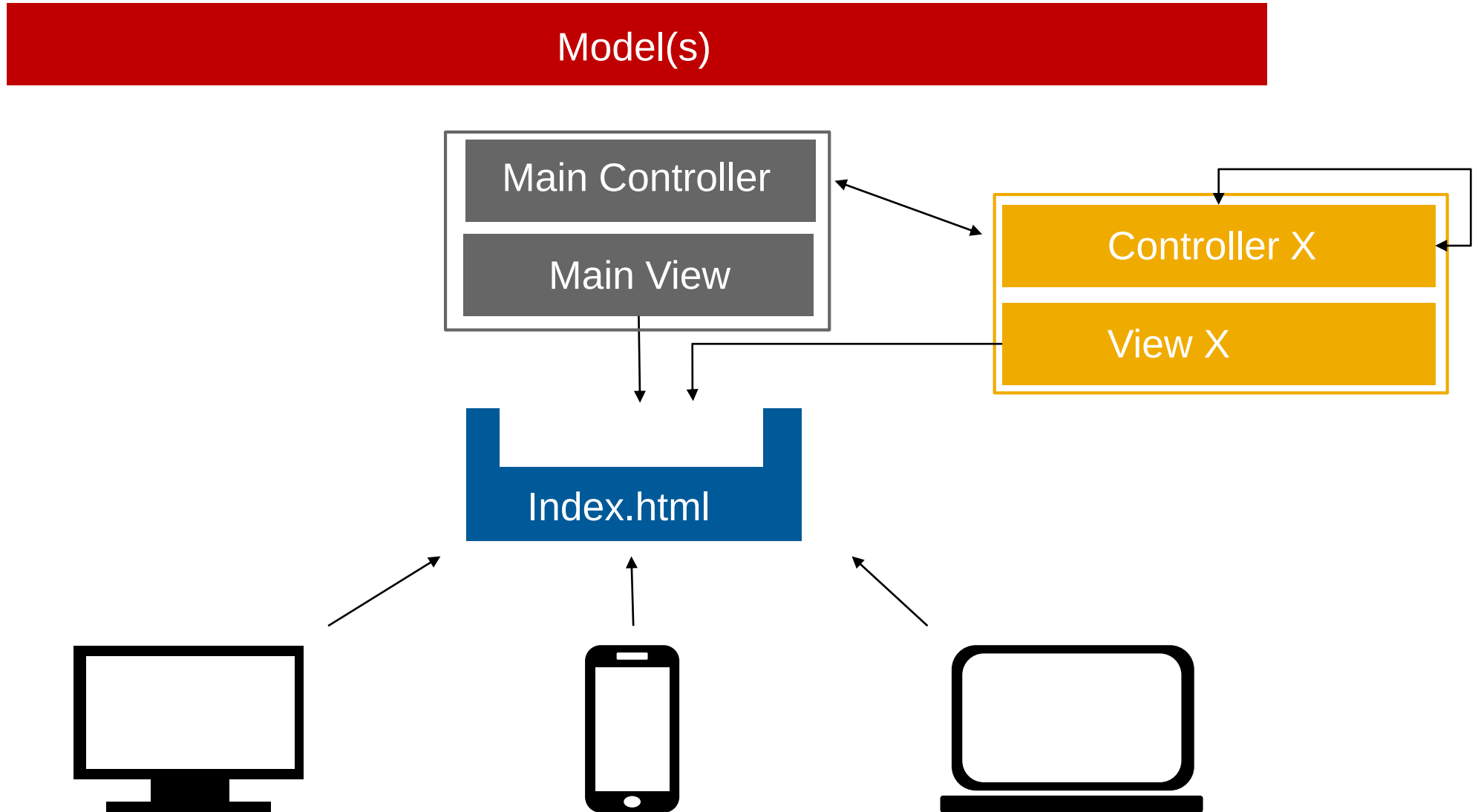
III. Model-View-Controller

Architectural pattern



III. Model-View-Controller

SAP UI5 Architecture



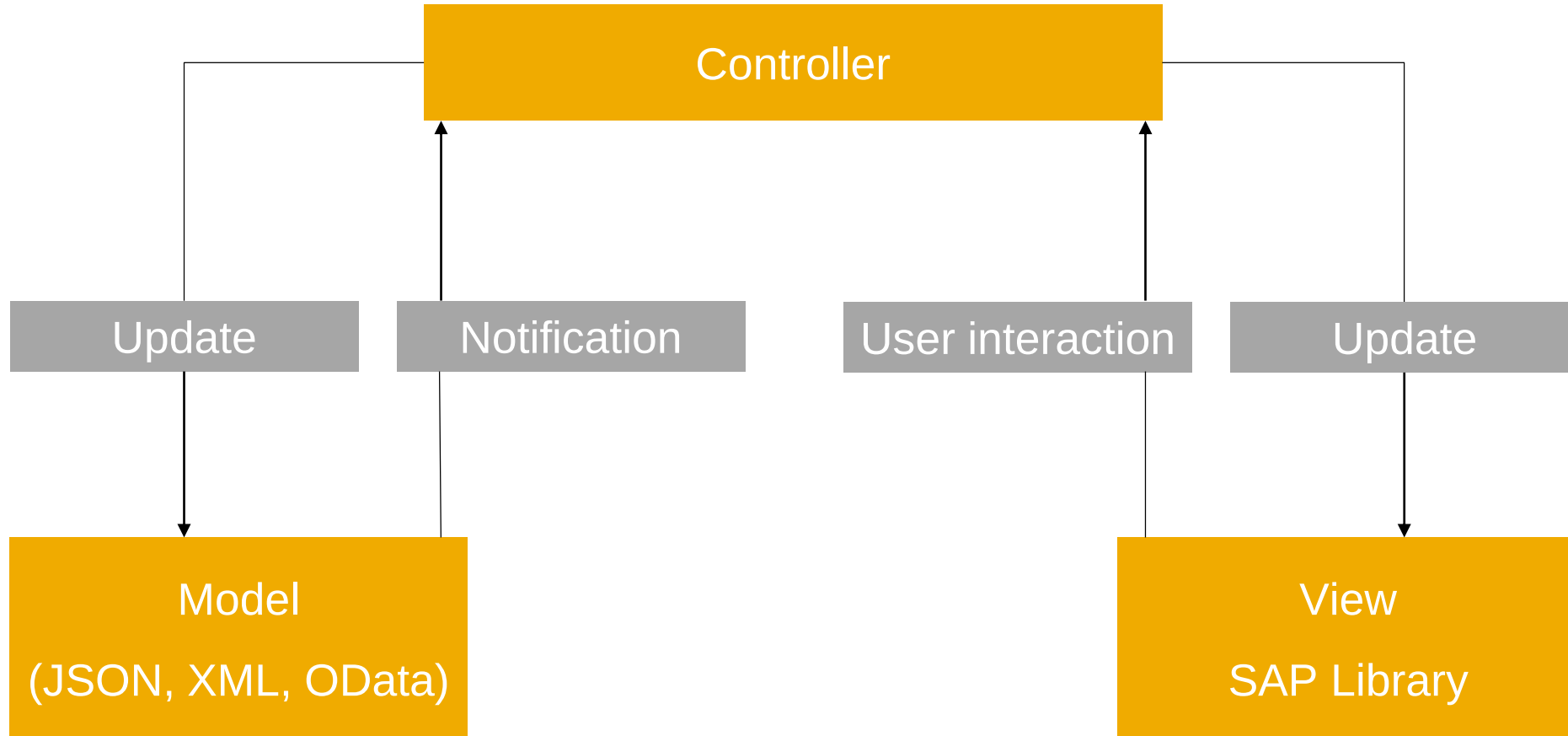
III. Model-View-Controller

The Model

- Represents the data layer and refers to the source of the application data
- Offers a possibility to access data with the common CRUD operations (create, read, update and delete)
- Often also known as *publisher*
- Predefined models:
 - ❖ JSON model (intended for small datasets, client-side)
 - ❖ XML model (intended for small datasets, client-side)
 - ❖ OData model (connection to server-side OData services)

```
{
  "Flight": [
    {
      "flightId" : "LH1234",
      "from" : "Munich",
      "to" : "Frankfurt",
      "date" : "11.05.2013",
      "time" : "16.50",
      "status" : "On time",
      "plannedArrival" : "16:45",
      "info" : "Arrives early"
    },
    {
      "flightId" : "LH2345",
      "from" : "Frankfurt",
      "to" : "Montreal",
      "date" : "23.12.2015",
      "time" : "11.20",
      "status" : "Delayed",
      "plannedArrival" : "12:20",
      "info" : "Delayed due to technical problems"
    },
    {
      "flightId" : "LH3456",
      "from" : "Montreal",
      "to" : "New York",
      "date" : "23.07.2016",
      "time" : "12.30",
      "status" : "Canceled",
      "plannedArrival" : "CANCELED",
      "info" : "Canceled due to bad weather"
    }
  ]
}
```

Architectural pattern in web development



III. Model-View-Controller

The Controller

- Controllers are always assigned to a specific view
- Controllers contain four **initial functions**:
 - **onInit**: Called when the controller is instantiated
 - **onBeforeRendering**: Called before the view is rendered
 - **onAfterRendering**: Called after the view is rendered
 - **onExit**: Called when the controller is destroyed



e.g. validation of
control elements

IV. Modularization & Navigation

Components

Independent and reusable parts in SAPUI5 applications

Are similar to classes in object-oriented languages

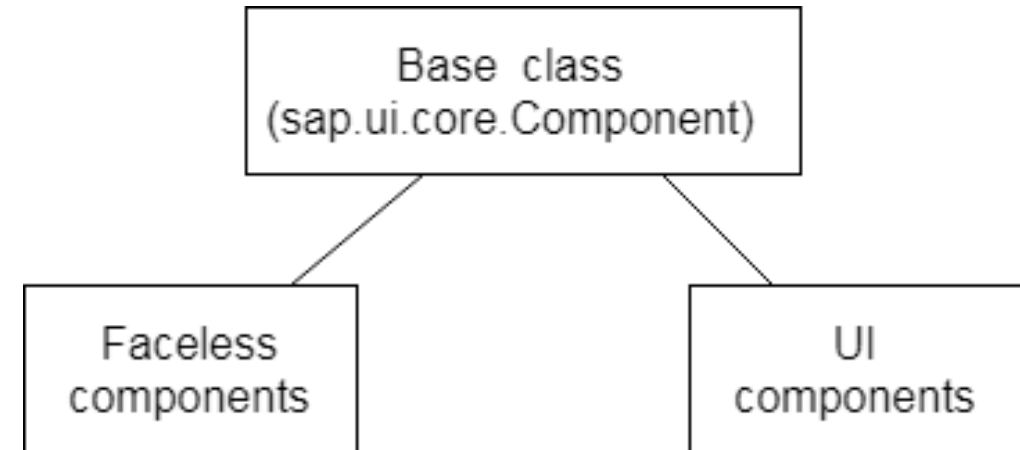
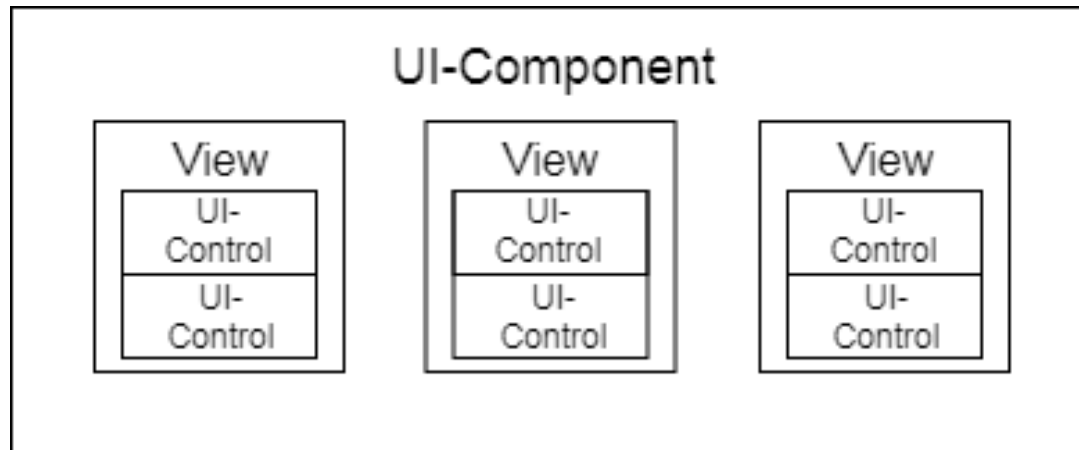
Advantages:

- Split application into smaller parts
- Can be loaded at runtime, when they are needed
- Can be developed independently
- Can be nested into one another or embedded into other applications (e.g. FIORI Launchpad)

IV. Modularization & Navigation

Component-types

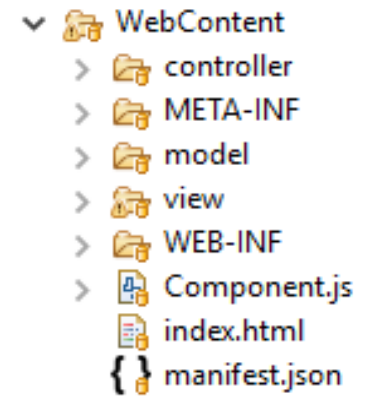
- Faceless components: Have no user interface, useful for backend-services
- UI components: Include multiple views, represent screen elements or areas on the UI



IV. Modularization & Navigation

Component structure

- Organized in unique namespace (all required and optional resources have to be organized in the namespace of the component)
- Consists of a component controller (Component.js, mandatory) and a descriptor (manifest.json)
- **Component.js:**
provides the runtime metadata and the component methods
- **Manifest.json:**
provides metadata, dependencies, configuration settings
- Optional: Views, controller, ...



Include (UI-)component in application

To render UI-components they have to be wrapped in a component container

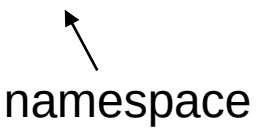
Component container:

- Separates the application and the nested component
- Can contain multiple components
- Carries specific settings and contains the lifecycle methods of a regular control (onBeforeRendering and onAfterRendering methods)

index.html

```
sap.ui.define(['jquery.sap.global', 'sap/ui/core/UIComponent'],
    function(jQuery, UIComponent) {
        "use strict";

        var Component = UIComponent.extend("sapui5.demo.mvcapp.Component",
            {
                metadata : {
                    manifest : "json"
                }
            });
        return Component;
    });
```



Component.js

```
sap.ui.getCore().attachInit(function()
{
    sap.ui.require([
        "sap/ui/core/ComponentContainer",
        "sapui5/mvcapp/Component"],
        function(ComponentContainer, Component){
            new ComponentContainer({
                height: "100%",
                component: new Component({
                    id: "mvcAppComponent"
                }).placeAt("content");
            });
        });
});
```

IV. Modularization & Navigation

Navigation

Navigation in sapui5

1. Shell based navigation
2. Event bus (asynchronous subscribe/publish mechanism, JavaScript events)
3. Routing (based on crossroads.js)

Intra-
application
navigation

Bookmark/
resume UI
states

Event bus:

- Communicate between active controllers
- Listens to every event



Router:

- Communicate between targets (e.g. view 1 and view 2)
- Listens only to browser url hash changes

IV. Modularization & Navigation

Routing

Advantages of routing (hash-based navigation):

- Use browser navigation to navigate between states and views
- Bookmark deep links and program states (even a specific search)
- Pass on data via the hash

IV. Modularization & Navigation

Routing

Traditional Web applications:

- server serves requested resources based on the URL pattern
- The server-side logic controls how the requested resource or page is displayed

Single-page applications:

- Only one page is initially requested from the server
- Additional resources are dynamically loaded using client-side logic
- The user only navigates within this page
- The navigation is persisted in the hash instead of the server path or URL parameters

IV. Modularization & Navigation

Routing

Config: define global configurations and default values that apply for all routes and targets

Routes: define a name, a pattern and targets to which to navigate when the route has been matched

They can be hard-coded or include optional, mandatory or query parameters

Targets: defines the view that is displayed. Needs target key and view name

```
//Router is globally available within the component
var oRouter = this.getRouter();
//Initialize router
oRouter.initialize();
//Trigger navigation
oRouter.navTo("targetname",{});
```

```
"routing": {
  "config": {
    "routerClass": "sap.m.routing.Router",
    "viewType": "XML",
    "viewPath": "sap.ui.demo.comp1.view",
    "controlId": "app",
    "controlAggregation": "pages",
    "transition": "slide",
    "async": true
  },

```

```
"routes": [{
  "pattern": "",
  "name": "appHome",
  "target": "home"
}],
"targets": {
  "home": {
    "viewId": "home",
    "viewName": "Home",
  }
}
```

Now you know the Model-View-Controller concept of SAPUI5 applications.

To consolidate your knowledge, you can do task 4 and 5 of the SAPUI5 exercise.



Debugging tools in SAPUI5

- Debugging is an important task in application development with regard to retracing program code and finding errors
- There is no central tool to debug SAPUI5 applications because they can be run on different platforms
- It is recommended to use developer tools of the browser to debug SAPUI5 applications
- Every modern web browser (e.g. Google Chrome, Mozilla Firefox, Internet Explorer) has an integrated debugger in its developer tools

IV. Debugging SAPUI5 applications

Debugging process in SAPUI5

Flight Monitor

Arrival Departure

Flight ID	From	To	Date	Time	Status	
LH 1234	Munich	Frankfurt	11.05.2013	16.50	On time	>
LH 2345	Frankfurt	Montreal	23.12.2015	11.20	Delayed	>
LH 3456	Montreal	New York	23.07.2016	12.30	Canceled	>

Elements Console Sources Network Performance Memory Application Security Audits

top Filter Default levels Group similar

2018-04-24 12:43:28 Device API logging initialized - DEVICE

2018-04-24 12:43:28 registerResourcePath ('', 'resources/') - sap.ui.ModuleSystem

2018-04-24 12:43:28 URL prefixes set to: - sap.ui.ModuleSystem

2018-04-24 12:43:28 (default) : resources/ - sap.ui.ModuleSystem

▶ [Deprecation] Synchronous XMLHttpRequest on the main thread is deprecated because of its detrimental effects to the page. For more information, see the specification: <https://xhr.spec.whatwg.org/>.

2018-04-24 12:43:30 registerResourcePath ('mvc_app', './mvc_app/') - sap.ui.ModuleSystem

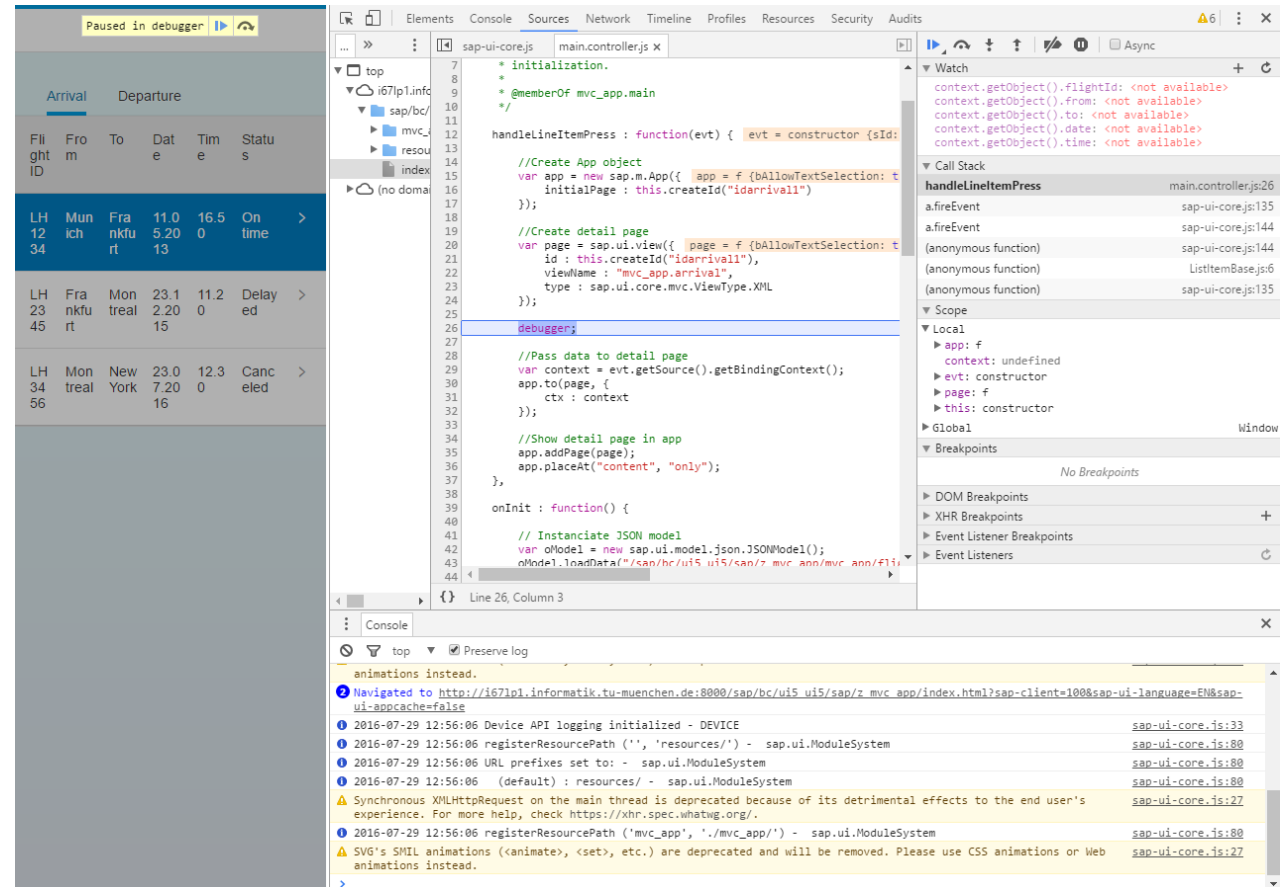
> |

- The **console** is a central tool to find strong program errors (e.g. syntax errors)

V. Debugging SAPUI5 applications

Debugging process in SAPUI5

- The console is a central tool to find strong program errors (e.g. syntax errors)
- Setting breakpoints can help you to retrace the program execution and variable values at certain points in the program
- Always make use of the debugger if you want to know more about your implemented program logic



IV. Debugging SAPUI5 applications

Debugging process in SAPUI5

Flight Monitor

Arrival **Departure**

Flight ID	From	To	Date	Time	Status	
LH 1234	Munich	Cluj	11.05.2013	16.50	On time	>
LH 2345	Frankfurt	Munich	23.12.2015	11.20	Delayed	>
LH 3456	Berlin	Venice	23.07.2016	12.30	Canceled	>

Network

Filter: Hide data URLs **All** XHR JS CSS Img Media Font Doc WS Manifest Other

10 ms 20 ms 30 ms 40 ms 50 ms 60 ms 70 ms 80 ms

Name: ☐ departures.json

× Headers Preview Response Cookies Timing

```
{DepartureFlight: [...]}
▼ DepartureFlight: [...]
  ▶ 0: {flightId: "LH1234", from: "Munich", to: "Cluj", date: "11.05.2013", time: "16.50", status: "On time",...}
  ▶ 1: {flightId: "LH2345", from: "Frankfurt", to: "Munich", date: "23.12.2015", time: "11.20",...}
  ▶ 2: {flightId: "LH3456", from: "Berlin", to: "Venice", date: "23.07.2016", time: "12.30",...}
```

- The tab **Network** provides you with information about the requested data
- Have a look on it in case you have troubles with your **data model**

Now you know how to debug SAPUI5 applications.

To consolidate your knowledge, you can do task 6 of the SAPUI5 exercise.



Check your knowledge

Check your knowledge

- The index.html file has to be modified in order to show an initial view.
☐ True ☐ False
- Which statement(s) about the Model-View-Controller concept is/are correct?
☐ Models are created within a controller
☐ Models can be directly assigned to views
☐ Controllers pass the model to the view
- With which programming languages can model, view and controller be implemented within an SAPUI5 application?
- A text field in your application is empty although you created a reference to a data element. How would you debug your application to find out the reason for this?

Solution

Solution

- The index.html file has to be modified in order to show an initial view.
☐ True ☒ False
- Which statement(s) about the Model-View-Controller concept is/are correct?
☒ Models are created within a controller
☐ Models are directly referenced within views
☒ Controllers pass the model to the view
- With which programming languages can model, view and controller be implemented within an SAPUI5 application?
See section *Creating and implementing SAPUI5 applications*
- A text field in your application is empty although you created a reference to a data element. How would you debug your application to find out the reason for this?
See section *Debugging SAPUI5 applications*

References

- Antolovic, M.: (2016): Einführung in SAPUI5, SAP PRESS 2016

© 2018 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. Please see <http://global12.sap.com/corporate-en/legal/copyright/index.epx> for additional trademark information and notices.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors.

National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP SE or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP SE or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

In particular, SAP SE or its affiliated companies have no obligation to pursue any course of business outlined in this document or any related presentation, or to develop or release any functionality mentioned therein. This document, or any related presentation, and SAP SE's or its affiliated companies' strategy and possible future developments, products, and/or platform directions and functionality are all subject to change and may be changed by SAP SE or its affiliated companies at any time for any reason without notice. The information in this document is not a commitment, promise, or legal obligation to deliver any material, code, or functionality. All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Readers are cautioned not to place undue reliance on these forward-looking statements, which speak only as of their dates, and they should not be relied upon in making purchasing decisions.